

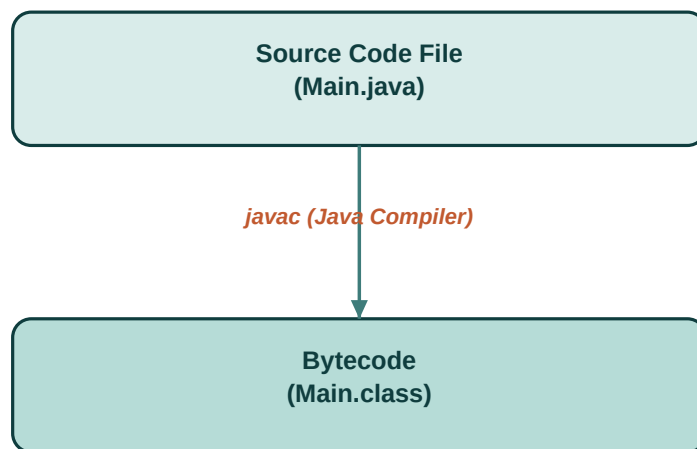
Java Execution Architecture & Platform Independence

Compilation, the JVM, and Write-Once-Run-Anywhere

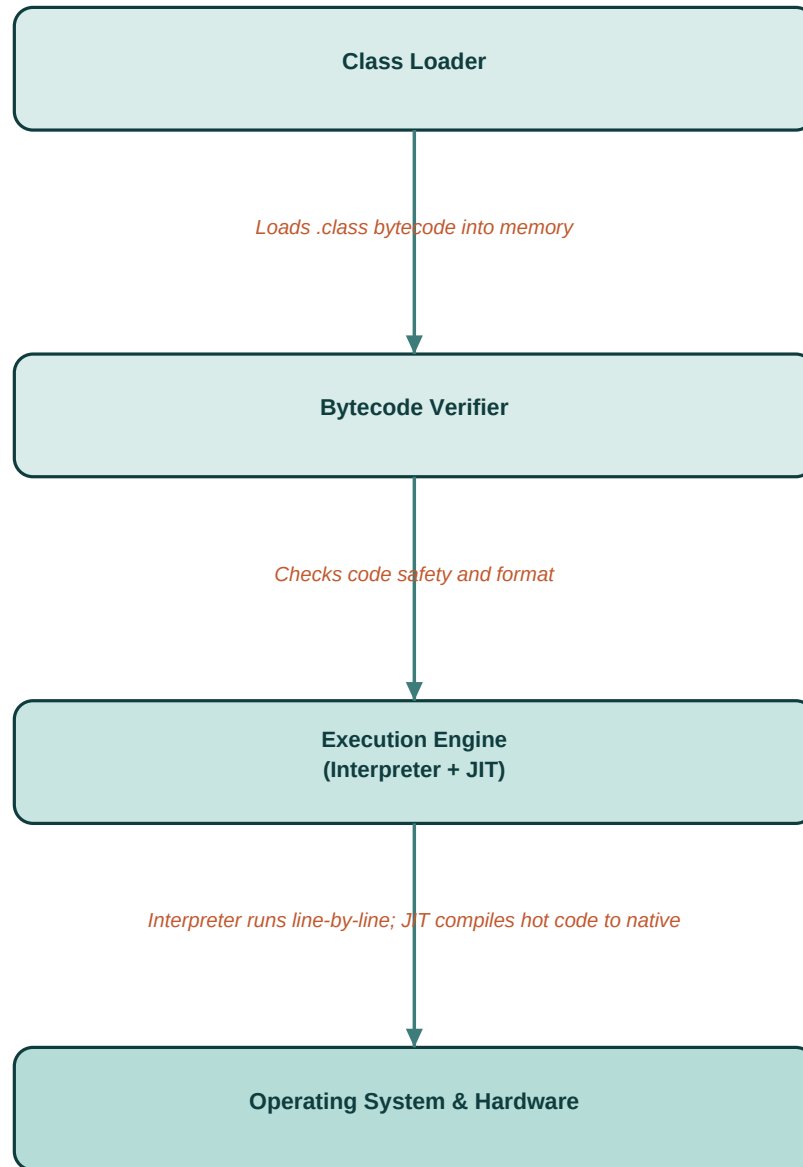
1. How Java Code Executes

Java uses a two-step execution model: it is both **compiled** and **interpreted**.

Compile Time



Runtime: JVM Execution



Bytecode flows from the Class Loader, through verification, into the Execution Engine, which runs it on the underlying OS and hardware.

Execution Steps

Step	What Happens
Compile Time (javac)	The Java compiler checks the source code (.java) for syntax errors and converts it into an intermediate, universal format called Bytecode (.class).
Loading (Class Loader)	The JVM loads the compiled .class file into main memory.
Verification (Bytecode Verifier)	The JVM verifies that the bytecode is valid, structurally safe, and does not violate memory access rules.
Execution — Interpreter	Reads bytecode line by line and converts it directly into machine code for the OS.
Execution — JIT Compiler	Detects frequently repeated code (hotspots/loops) and compiles it into native machine code once so it runs at hardware speed without repeated interpretation.

2. Platform Independence (WORA)

Definition: Platform independence means Java code can be written on one operating system and run on any other operating system without modification: “Write Once, Run Anywhere” (WORA).

Why Java Is Platform Independent

	Reason
1	The Java compiler produces universal Bytecode (.class), not OS-specific machine instructions.
2	Any operating system with a compatible JVM can read and execute this bytecode.

Important Distinction

Component	Nature
Java Bytecode	Platform independent — identical on all machines.
Java Virtual Machine (JVM)	Platform dependent — Windows, Linux, and macOS each require their own distinct JVM build.

Real-World Analogy: A book written in Braille. The text (Bytecode) is universal. Any trained reader in India, the US, or Japan (the JVM for that region) can read the exact same Braille page and translate it into their native spoken tongue.

JAVA

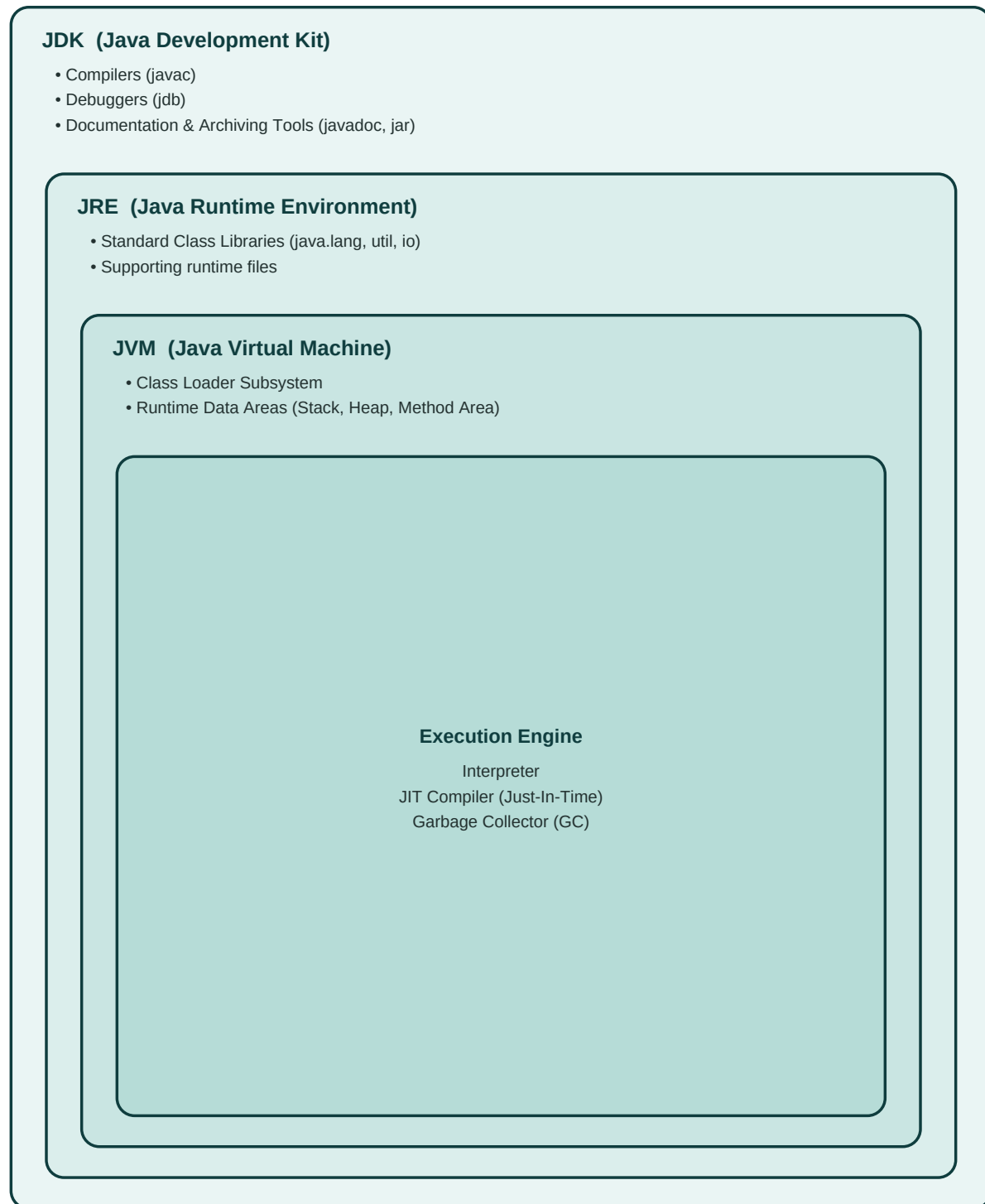
```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Execution Flow in Terminal

BASH

```
javac HelloWorld.java    # Produces HelloWorld.class (Bytecode)
java HelloWorld          # JVM executes bytecode on any OS
```

3. Relationship: JDK vs. JRE vs. JVM vs. JIT



Each layer contains the one inside it: the JDK includes everything in the JRE, which includes everything in the JVM, whose Execution Engine does the actual work of running bytecode.

4. Component Breakdown in Detail

1. JVM (Java Virtual Machine)

What it is: The abstract software engine that executes Java bytecode on target hardware.

Core Duties

	Duty
1	Allocates Stack and Heap memory.
2	Loads, verifies, and runs .class files.
3	Manages memory automatically via the Garbage Collector.

2. JIT (Just-In-Time Compiler)

What it is: A specialized performance-optimization component inside the JVM's Execution Engine.

How It Works

	Step
1	The Interpreter runs code line by line from the start.
2	If a method or loop runs repeatedly ("hot code"), the JIT Compiler converts that entire block directly into native machine code.
3	Subsequent executions call the native binary directly, eliminating interpretation overhead.

Real-World Analogy: An interpreter in a meeting translating speech word-by-word. When the speaker repeats the company mission statement for the tenth time, the interpreter hands out a pre-printed translated card instead of re-translating sentence-by-sentence.

3. JRE (Java Runtime Environment)

What it is: The software package needed only to run existing Java programs.

Contains: JRE = JVM + Core Class Libraries (e.g., java.lang, java.util) + Supporting Files.

Target Audience: End-users running compiled Java applications without writing or compiling code.

4. JDK (Java Development Kit)

What it is: The complete software development environment needed to create, compile, and run Java applications.

Contains: JDK = JRE + Development Tools (javac, javadoc, jdb, jar).

Target Audience: Software developers and programmers.

Comparison Summary

Component	Full Form	Purpose	Includes
JVM	Java Virtual Machine	Runs bytecode on a specific OS.	Class Loader, Execution Engine, Memory Areas.
JIT	Just-In-Time Compiler	Speeds up execution of repeated bytecode.	Included inside the JVM Execution Engine.
JRE	Java Runtime Environment	Minimum requirement to run Java apps.	JVM + Standard Class Libraries.
JDK	Java Development Kit	Complete toolkit to develop & run apps.	JRE + Development Tools (javac, debugger).